

Programming Assignment 3: Ensembles, Generative Models, & RL

PRNN 2026

Department of Electrical Communication Engineering, IISc

Prathosh AP

Overview & General Instructions

In this final assignment, you will build powerful ensemble methods for tabular data, abandon explicit labels to learn underlying data distributions, generate novel samples, and train active agents.

Rules of Engagement:

- **No Black Boxes:** You must implement core mathematical algorithms (e.g., Gini splits, AdaBoost reweighting, InfoNCE, ELBO, REINFORCE) from scratch using Python/NumPy/PyTorch. `sklearn` ensemble wrappers are strictly forbidden.
- **Tracking for Diagnostic Exam:** You are expected to record exact numerical metrics. A closed-book diagnostic quiz will follow this assignment.
- **Deliverables:** Submit a 4-page maximum PDF report containing the requested plots, mathematical explanations, and metrics, along with your heavily commented code.

The Datasets & Modalities

You will use the exact same splits generated in Assignment 2 for the following datasets:

1. **Delhi Air Quality Time-Series:** Used for Tree Ensembles, PCA, and RL Environments.
2. **PlantVillage Crop Disease:** Used for VAEs, GANs, and Self-Supervised Learning.
3. **APTOS 2019 Blindness Detection:** Used for Advanced GAN Evaluation (FID).

1 Phase 1: Tree Ensembles & Boosting (Air Quality Data)

For Questions 1.1 to 1.3, convert the PM2.5 target into a binary classification task: 1 if $PM_{2.5} > 200$, else 0.

- 1.1 Information Gain from Scratch:** Write a Python function to compute the exact Gini Impurity reduction for a continuous feature threshold. Iterate over the Air Quality “Temperature” feature. Report the exact optimal numerical threshold (split value) that maximizes the Gini gain for the root node of a decision tree.

1.2 Random Forest: Implement a Random Forest classifier using an ensemble of depth-limited trees with feature bagging (random subspaces).

1.3 AdaBoost Sample Reweighting: Implement AdaBoost (AdaBoost.M1) using shallow trees. Track the sample weights of the 5 most consistently misclassified training samples over 50 boosting iterations. Plot their exponential weight growth and mathematically explain the AdaBoost update rule driving this explosion.

1.4 Gradient Boosting Residuals (Regression): Revert to the continuous PM2.5 target. Implement a basic Gradient Boosted Regressor by sequentially fitting shallow trees to the pseudo-residuals. Plot the statistical variance of the training residuals after 1, 5, 10, and 50 boosting stages.

2 Phase 2: Subspace & Clustering (Air Quality & PlantVillage)

2.5 PCA vs. Linear Autoencoders: Flatten the 72-hour Air Quality sequence. Implement PCA via Singular Value Decomposition (SVD) and compute the exact number of components required to retain 95% of the variance. Train a purely linear Autoencoder (no non-linear activations) with a bottleneck dimension equal to this component count. Prove empirically (via reconstruction MSE) that the AE spans the exact same subspace as PCA.

2.6 GMM Latent Clustering: Extract the final dense feature representations of the PlantVillage validation set using your trained CNN from Assignment 2 (strip the classification head). Fit a Gaussian Mixture Model (GMM) on these embeddings. Compute the Adjusted Rand Index (ARI) between your unsupervised GMM clusters and the true disease labels.

3 Phase 3: Deep Generative Models (PlantVillage & APTOS)

3.7 ELBO Implementation: Build a Convolutional VAE for the PlantVillage dataset. Plot both the reconstruction and regularization terms on the same graph over your training epochs.

3.8 VAE vs. Simple DCGAN: Build and train a simple Deep Convolutional GAN (DCGAN) on the same PlantVillage dataset. Generate a grid of 16 synthetic leaf images from both your trained VAE and your DCGAN. Compare the visual crispness and mathematically explain why the VAE ELBO objective inherently produces blurrier images compared to the GAN minimax objective.

3.9 Fréchet Inception Distance (FID): Filter the APTOS dataset to contain *only* Class 4 images. Train a DCGAN on this subset. Write a script to pass 500 real APTOS images and 500 of your GAN images through a frozen `torchvision.models.inception_v3`. Extract the 2048-dimensional features. Implement the FID formula using SciPy matrix operations and report your exact FID score.

3.10 Batch Normalization & Mode Collapse: Remove all Batch Normalization layers from your APTOS Generator network and retrain from scratch. Provide a visual grid of the resulting outputs and mathematically explain how the lack of batch statistics fundamentally exacerbates mode collapse.

4 Phase 4: Self-Supervised Learning via SimCLR (PlantVillage)

4.11 InfoNCE Loss: The heart of SimCLR is the contrastive data augmentation pipeline (random crops, flips, color jitter). Implement the **Normalized Temperature-scaled InfoNCE loss from scratch**. Train a ResNet18 encoder and an MLP projection head on the PlantVillage dataset using *only* this contrastive loss (ignore labels). **Report the final loss on the validation set.**

4.12 The Linear Evaluation Protocol: Freeze the weights of your SimCLR encoder. Attach a single, untrained `nn.Linear` classification head. Train *only* this linear head using the true PlantVillage labels. **Compare the test accuracy of this self-supervised approach to the fully supervised CNN you built in Assignment 2.**

5 Phase 5: Policy Gradient Theorem (Air Quality RL)

Environment Setup: We formulate an Air Quality MDP. **State:** The past 24 hours of meteorological data. **Action:** $a \in \{0, 1\}$ (Turn Purifier OFF or ON). **Reward Function:** $R = -0.1$ if $a = 1$ (energy cost). If $a = 0$ and the actual $t + 1$ PM2.5 > 150 , $R = -10.0$ (health penalty). Else, $R = 0.0$.

5.13 REINFORCE Implementation: Implement the Monte Carlo Policy Gradient (REINFORCE) algorithm from scratch. Build a Policy Network $\pi_\theta(a|s)$ that outputs the probability of turning the purifier ON. Simulate trajectories through the training time-series and update θ using the log-derivative trick. Plot the total episodic return over 500 episodes.